College of Computer Science and Information Technology
Department of Computer Engineering

ARTI 404 – Image Processing

Lab 3 Report: Geometric and Intensity Transformations

Student Name Mohammed Omar Halawani

Student ID 2240001174

Academic Year 2025 – 2026

1. Objective

This lab applies geometric transformations (scaling, rotation and shearing) and basic intensity transformations (negative, logarithmic and power-law) to a grayscale image using OpenCV and NumPy.

2. Image Used

- **cameraman.tif** – grayscale image, loaded from the `images/` folder.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("images/cameraman.tif", cv2.IMREAD_GRAYSCALE)

if img is None:
    raise FileNotFoundError("Image could not be loaded.")
```

3. Task 1 – Geometric Transformations

Scaling (2× enlargement)

The image is resized to twice its width and height with bilinear interpolation (`cv2.INTER_LINEAR`). Each new pixel value is a weighted average of the 4 nearest original pixels.

```
height, width = img.shape

resized = cv2.resize(
    img,
    (width * 2, height * 2),
    interpolation=cv2.INTER_LINEAR
)

plt.imshow(resized, cmap="gray", vmin=0, vmax=255)
plt.title("2x Enlarged Image")
plt.axis("off")
plt.show()
```



Figure 1: Image enlarged 2× with bilinear interpolation.

Discussion: The enlarged image keeps the same content at twice the resolution. Bilinear interpolation gives smooth results without the blocky look of nearest-neighbour interpolation, but it cannot add new detail, so fine edges look slightly softer.

Rotation (120°)

`cv2.getRotationMatrix2D` builds the affine matrix for a 120° rotation about the image centre with scale 1.0. A positive angle in OpenCV means counter-clockwise. `cv2.warpAffine` then applies it.

```
height, width = img.shape

center = (width / 2, height / 2)

rotation_matrix = cv2.getRotationMatrix2D(
    center,
    120,
    1.0
)

rotated = cv2.warpAffine(
    img,
    rotation_matrix,
    (width, height)
)

plt.imshow(rotated, cmap="gray", vmin=0, vmax=255)
plt.title("Rotated 120 Degrees")
plt.axis("off")
plt.show()
```

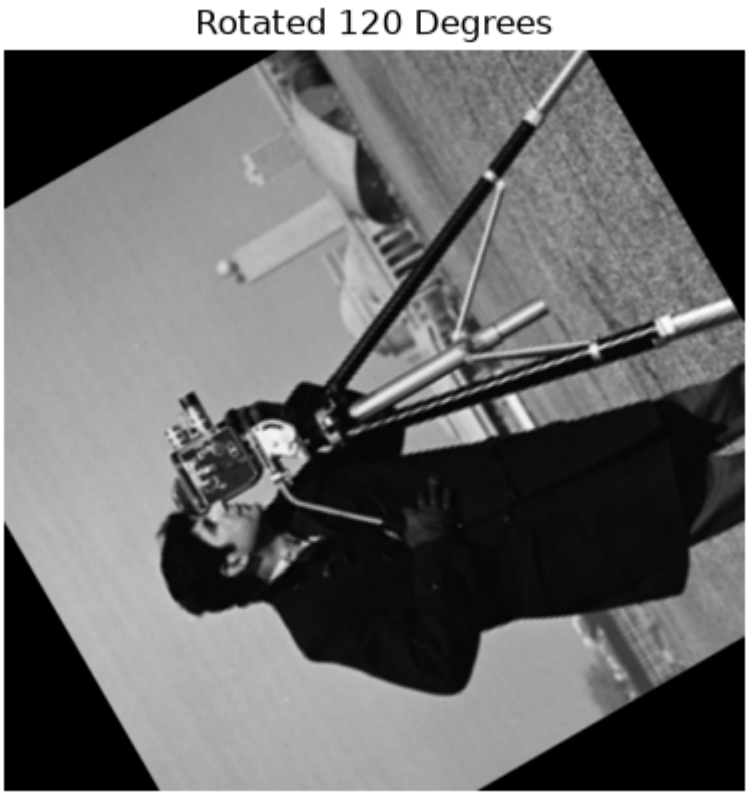


Figure 2: Image rotated 120° about its centre.

Discussion: The output size stays the same as the input, so the corners of the rotated image are cut off. The areas with no source pixel are filled with black. To keep the whole image, the output canvas would have to be enlarged and the translation part of the matrix adjusted.

Shearing

A horizontal shear with factor 0.5 uses the matrix [[1, 0.5, 0], [0, 1, 0]], so x' = x + 0.5·y. The output width is increased by 0.5 × height so that the sheared image fits.

```
height, width = img.shape

shear_factor = 0.5

shear_matrix = np.float32([
    [1, shear_factor, 0],
    [0, 1, 0]
])

sheared = cv2.warpAffine(
    img,
    shear_matrix,
    (int(width + shear_factor * height), height)
)

plt.imshow(sheared, cmap="gray", vmin=0, vmax=255)
plt.title("Sheared Image")
plt.axis("off")
plt.show()
```

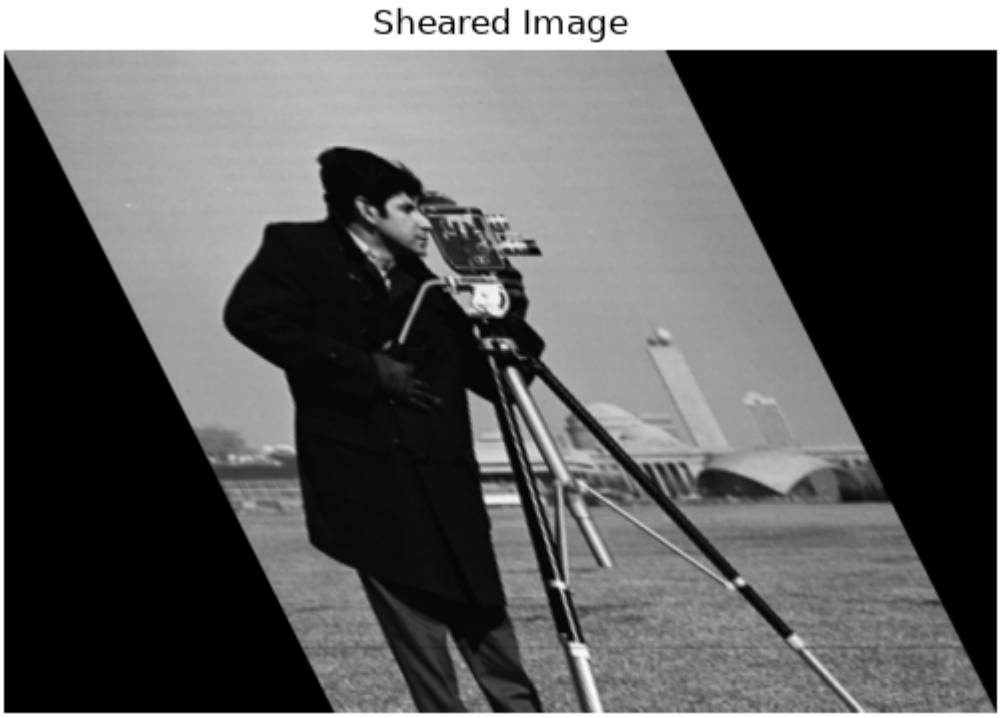


Figure 3: Image sheared horizontally with factor 0.5.

Discussion: Each row is shifted to the right by an amount that grows with its row number, which turns the rectangle into a parallelogram. The top rows barely move and the bottom rows move the most, which leaves black triangles on the left and right.

4. Task 2 – Intensity Transformations

Image negative

s = 255 – r

```
negative = 255 - img

plt.imshow(negative, cmap="gray", vmin=0, vmax=255)
plt.title("Negative Image")
plt.axis("off")
plt.show()
```

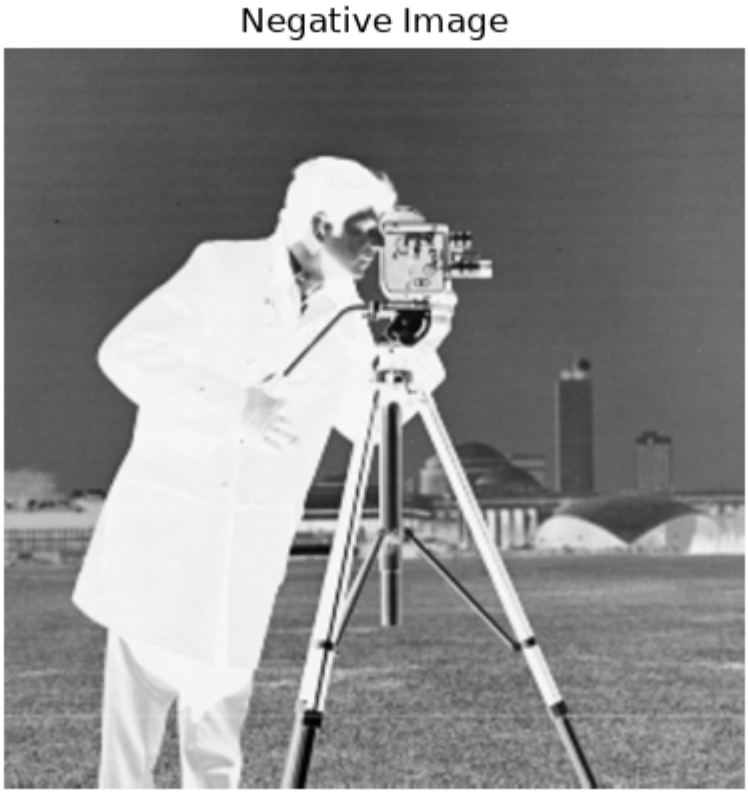


Figure 4: Negative of the image.

Discussion: The negative reverses the gray scale: the dark coat becomes white and the bright sky becomes dark. Negatives make detail in dark regions easier to see, such as the folds of the coat.

Log transformation

s = c · log(1 + r), with c = 255 / log(256) so that the output also covers 0–255.

```
c = 255 / np.log(256)

log_image = c * np.log(1 + img.astype(np.float32))
log_image = np.uint8(log_image)

plt.imshow(log_image, cmap="gray", vmin=0, vmax=255)
plt.title("Log Transformation")
plt.axis("off")
plt.show()
```



Figure 5: Log transformation.

Discussion: The log curve stretches the dark gray levels and compresses the bright ones. The dark coat and tripod show more internal detail, but the whole image becomes brighter and looks washed out and lower in contrast, because the mid and high tones are squeezed into a narrow range near white.

Power-law (gamma) transformation

$s = 255 \cdot (r / 255)^{\gamma}$, with $\gamma = 0.5$.

```
gamma = 0.5

normalized = img / 255.0

power_image = np.power(normalized, gamma)

power_image = np.uint8(power_image * 255)

plt.imshow(power_image, cmap="gray", vmin=0, vmax=255)
plt.title("Power-Law Transformation (Gamma = 0.5)")
plt.axis("off")
plt.show()
```



Figure 6: Power-law transformation with γ = 0.5.

Discussion: With γ < 1 the curve also brightens dark regions, but less strongly than the log transform. The image is lighter than the original, and the coat and shadows show more detail, but the overall contrast stays much better than with the log transform. A γ greater than 1 would do the opposite and darken the image.

5. Conclusion

Geometric transformations change where pixels are placed, using affine matrices with `cv2.warpAffine`, and the choice of output size decides whether parts of the image are clipped. Intensity transformations change pixel values only. The negative inverts the image,

while the log and gamma ($\gamma < 1$) transforms both bring out dark detail. Gamma correction gives finer control because the strength of the effect can be tuned.